

コンピュータアーキテクチャI

担当：佐々木 敬泰

第6回

メモリアクセス命令

今日の内容

- メモリアクセス命令
 - ロード命令
 - ストア命令
- 整列化制約

復習

- MIPSを含むRISC系プロセッサでは、
 - 演算はレジスタ間のみ
 - メモリとのデータ交換は専用命令を用いる

【発展】

IntelやAMDのプロセッサはCISC系命令セットを採用しており、演算命令がメモリと直接データをやりとりできる

例) `edx`番地の値を`eax`レジスタに足し込む

```
addl (%edx), %eax
```

```
# eax = eax + メモリ[edx]
```

ワード単位の読み出し

- `lw $1, 12($2)` (load word)
 - メモリから1語(4B=32bit)読み出し
 - `$1 = メモリ[$2 + 12]`

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

メモリアクセス命令

メモリアクセス命令

- メモリアクセス命令
 - メモリからの読み出し
 - メモリへの書き込み
- ⇒2個しかない???
- 種類
 - データサイズ (8bit/16bit/32bit)
 - 符号の有無

半語単位の読み出し

- `lh $1, 12($2)` (load half)
 - メモリから半語(2B=16bit)読み出し
 - `$1 = メモリ[$2 + 12]`
 - 読み出された値は符号拡張される
- `lhu $1, 12($2)` (load half unsigned)
 - メモリから半語(2B=16bit)読み出し
 - `$1 = メモリ[$2 + 12]`
 - 読み出された値は符号なしで拡張される

バイト単位の読み出し

- **lb \$1, 12(\$2)** (load byte)
 - メモリから1バイト(8bit)読み出し
 - \$1 = メモリ[\$2 + 12]
 - 読み出された値は符号拡張される
- **lbu \$1, 12(\$2)** (load byte unsigned)
 - メモリから1バイト読み出し
 - \$1 = メモリ[\$2 + 12]
 - 読み出された値は符号なしで拡張される

9

エンディアン (バイトオーダー)

- 上位桁の値から書くか、下位桁の値から書くかを定義するもの

番地	0	1	2	3
0	09	21	13	4F
4	3C	5A	BF	39
8	01	23	45	67
12	DE	AD	BE	EF
...	...	...	...	...

- ビッグエンディアン
 - 上位桁を先に書く
- リトルエンディアン
 - 下位桁を先に書く
- \$1=8のときlw \$2, 0(\$1)を考える
 - ビッグエンディアン: \$2=0x01**234567**
 - リトルエンディアン: \$2=0x**674523**01

2バイト以上のデータを扱う場合は要注意!

11

事例(1/2)

- パソコン用のアプリケーション・ソフトウェアを長年開発してきたPさん
- ある開発プロジェクトで、音響機器のオーディオ・ファイル生成する組み込みソフトウェア・モジュールを設計
- オーディオ・ファイルにはパソコンでも取り扱いができるフォーマットを採用していたため、Pさんは設計したモジュールの単体テストをすべてパソコン上でを行い、結合テストに臨みました
- このモジュールをターゲット・システムに組み込んでオーディオ・ファイルを生成し、パソコンに取り込ませようとしたところ、ファイルとして全く認識できないことが判明

<https://xtech.nikkei.com/dm/article/NEWS/20080220/147695/>

13

ワード単位の書き込み

- **sw \$1, 12(\$2)** (store word)
 - メモリに1語(32bit)書き込み
 - メモリ[\$2 + 12] = \$1

15

アドレスと読み出される値

- メモリは基本的に32bit(4B)単位で考える (32bit CPUの場合)
- \$1=8とする
 - lw \$2, 4(\$1)
 - \$2=0xdeadbeef
 - lh \$2, 4(\$1)
 - \$2=0xffffdead
 - lhu \$2, 4(\$1)
 - \$2=0x0000dead
 - lb \$2, 4(\$1)
 - \$2=0xfffffde
 - lbu \$2, 4(\$1)
 - \$2=0x000000de

番地	0	1	2	3
0	09	21	13	4F
4	3C	5A	BF	39
8	01	23	45	67
12	DE	AD	BE	EF
...	...	...	...	...

10

エンディアンの違い

- プロセッサ

プロセッサ	エンディアン
インテル/AMD x86(x64)	リトル
PowerPC	ビッグ
SPARC	ビッグ
MIPS	両対応 (ビッグが多い)
ARM	両対応

- データ

種類	エンディアン
BMP (画像データ)	リトル
USB	リトル
TCP/IP (ネットワーク)	ビッグ

ソフトウェアを書く上でも重要!

12

事例(2/2)

- 原因

- パソコンとターゲット・システムのエンディアンが異なっていた!

図10-2 ヘッド情報のダンプ結果の比較

オフセット	+0	+1	+2	+3	+4	+5	+6	+7
パソコン上でメモリをダンプ	56	34	12	00	FF	FF	23	01
ターゲット・システム上でメモリをダンプ	00	12	34	56	01	23	FF	FF

オーディオデータのサイズ情報(サイズが0x00123456だった場合)

- 同様の事例は非常に多い

エンディアンは、
・プロセッサ依存
・データの内部情報も!

14

半語単位の書き込み

- **sh \$1, 12(\$2)** (store half)
 - メモリに半語(16bit)書き込み
 - メモリ[\$2 + 12] = \$1
 - lhuはあるのにshuはないの?
 - 不要なのでない
- ⇒レジスタは32bit長なので符号の有無が結果に影響。メモリはバイト単位でアクセスできる。shはレジスタの下位16bitしか書き込まないため符号有無の区別がない。

16

バイト単位の書き込み

- **sb \$1, 12(\$2)** (store byte)
 - メモリに1バイト(8bit)書き込み
 - メモリ[\$2 + 12] = \$1
 - sbuはない (shと同様の理由)

➡ 次スライドで読み書きと
符号の関係をまとめる

17

C言語とアセンブラの対比

- C言語のソース

```
int x[4]; // 1語は4バイト
x[0] = x[1] + x[2];
```

- 変換後のアセンブラ

```
lw $2, 4($30)    # $2 = MEM[$30 + 4]
lw $3, 8($30)    # $3 = MEM[$30 + 8]
add $2, $2, $3    # $2 = $2 + $3
sw $2, 0($30)    # MEM[$30 + 0] = $2
※ $30は配列xの先頭アドレスと仮定
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
x[0]				x[1]				x[2]				x[3]			

19

メモリアクセス命令と符号

- ストア命令の場合

- レジスタ
 - \$1は0x01020304
- ストア命令
 - sw \$1, 20(\$0)

番地	20	21	22	23
値	01	02	03	04

ここは変更
されない

- sh \$1, 20(\$0)

番地	20	21	22	23
値	03	04	43	21

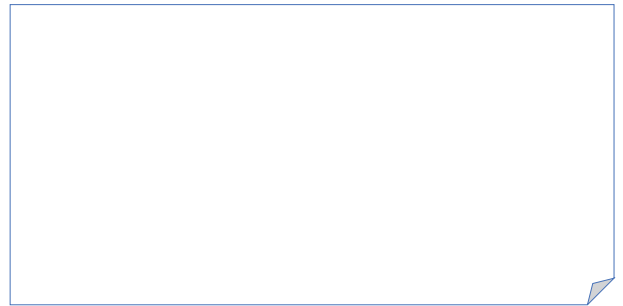
- sb \$1, 20(\$0)

番地	20	21	22	23
値	04	65	43	21

18

練習

- 10番地から12番地に入っている3つの符号つき
バイトデータの総和を計算するプログラムをア
センブラで記述する。



20

整列化制約

コンピュータアーキテクチャI

21

整列化制約(alignment restriction)

- ワードアドレスは4の倍数でなくてはならない

		メモリ				
	ワードアドレス	番地	0	1	2	3
	バイトセクタ	0	01	9d	1d	df
	読み出し命令	4	ba	98	1d	7f
	書き込み命令	8	31	09	ac	a1
	データ	12	67	12	d1	ba
		...	...	...	...	...

- 一般的なメモリは（構造上）ワード単位でしか
アクセスできない
- バイト単位の書き込みはバイトセクタを使う
 - 下位ビットのみ書き込むなら0001₂にする

23

ワード単位の読み出し

- **lw \$1, 12(\$2)** (load word)

- メモリから1語(4B=32bit)読み出し
- \$1 = メモリ[\$2 + 12]
- アドレス(上の例なら\$2+12)は必ず4の倍数でなくてはならない（違反時は実行時に例外が発生する）

追加

➡ 何故？

22

整列化制約のまとめ

- ワード単位
 - アドレスは4の倍数
- 半語（ハーフワード）単位
 - アドレスは2の倍数
- バイト単位
 - 任意の整数
- 整列化制約を満たしていない場合は、実行時に
エラー（例外）が発生する

コンピュータアーキテクチャI

24

境界を跨ぐメモリアクセス

- 1番地から3番地にある1ワードのデータがどうしても欲しい
- 解決方法
 1. バイト単位で読み出して、プログラム側で連結加工する
 2. 特殊な命令を使う
 - MIPS等、ごく一部のプロセッサ限定
 - 多くのプロセッサではサポートしていない

番地	0	1	2	3
0	00	01	02	03
4	04	05	06	07

25

境界を跨ぐメモリアクセス命令

- lwl/lwr* 命令 (MIPSの場合)

lwr \$2, 1(\$0) # 上位3Bをロード

lwl \$2, 1(\$0) # 下位1Bをロード

MIPS社の特許
(すでに時効)

番地	0	1	2	3
0	00	01	02	03
4	04	05	06	07

レジスタ	01	02	03	04
------	----	----	----	----

- 同様にswl/swr命令もある

* Load Word Left/Load Word Right

26